

CellRejuvo: Rescuing the Aging of 3D NAND Flash Cells with Dense-Sparse Cell Reprogramming

Han-Yu Liao^{1,4}, Yi-Shen Chen^{1,5}, Jen-Wei Hsieh^{1,6}, Yuan-Hao Chang^{2,7} and Hung-Pin Chen^{3,8}

¹National Taiwan University of Science and Technology, Taipei, Taiwan

²Institute of Information Science, Academia Sinica, Taipei, Taiwan

³Innodisk Corporation, New Taipei, Taiwan

⁴M11115005, ⁵yishen, ⁶jenwei}@mail.ntust.edu.tw, ⁷johnson@iis.sinica.edu.tw, ⁸hungpin_chen@innodisk.com

ABSTRACT

3D NAND flash memory is one of the most important storage technologies in modern computer systems because of its non-volatile nature and excellent data access performance. However, it suffers from aging and reliability issues due to its inherent property. In contrast to the previous research that tried to recover the data with additional encoding techniques, we propose a novel reprogramming technique, called CellRejuvo, to improve the reliability of NAND flash cells. To the best of our knowledge, CellRejuvo is the pioneer for data recovery technique that cleverly leverages reprogramming to alleviate cell aging, extending the lifespan of solid-state drives. We implement CellRejuvo on a real 3D NAND flash-based SSD and evaluate its capability on various realistic workloads. The extensive experimental results show that CellRejuvo successfully reduces the error rate of SSD by an average of 38.28% under various retention times.

KEYWORDS

3D NAND flash, reprogram, error recovery, reliability

ACM Reference Format:

Han-Yu Liao^{1,4}, Yi-Shen Chen^{1,5}, Jen-Wei Hsieh^{1,6}, Yuan-Hao Chang^{2,7} and Hung-Pin Chen^{3,8}. 2024. CellRejuvo: Rescuing the Aging of 3D NAND Flash Cells with Dense-Sparse Cell Reprogramming. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*, October 27–31, 2024, New York, NY, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3676536.3676703>

1 INTRODUCTION

3D NAND flash has gained an increasingly high position among mainstream storage media due to its high performance, shock resistance, and large storage capacity [15, 22, 27]. However, to meet the growing demand for storage capacity, some trade-offs have to be made. For example, the original NAND flash could store only one bit per cell, but as the technology advanced, the number of bits per cell increased. The flash architecture evolved from SLC to MLC, then to TLC, and even QLC devices, which can store 4 bits per cell. Although storing multiple bits per cell can provide larger

storage space, it reduces the margins between different states. The shortened margins make the NAND flash become less tolerant of the error effects caused by charge loss in NAND flash cells; in other words, the reliability of NAND flash decreases.

To ensure data reliability of flash memory, there are two main kinds of approaches for data recovery. Firstly, Error Correction Code (ECC) can detect and correct a certain number of error bits within a threshold of 120 error bits per 1K code word. The implementation of ECC contains Bose–Chaudhuri–Hocquenghem (BCH) and Low-Density Parity-Check (LDPC) encoding [10, 11]. BCH employs algebraic encoding to correct a fixed number of error bits [21], whereas LDPC utilizes a probabilistic approach to correct a greater number of random error bits, albeit at the expense of increased computational resources. ECC, on the other hand, is exclusively utilized to rectify errors in read data and does not address the inherent errors present in NAND flash cells. Secondly, the read retry mechanism is another approach that adjusts the read reference voltage during data retrieval to mitigate data errors [2]. When NAND flash storage devices read data, they utilize a read reference voltage value to assess the state of data cells. If the error counts of the result obtained from the read reference voltage exceed the upper limit that ECC can correct, the storage device proceeds to reattempt data retrieval using a different read reference voltage offset. If the correct result cannot be obtained after a certain number of retries, the storage device will report an error [19]. Note that in practical applications, the extent of data errors and voltage distribution remains unknown. Therefore, the read retry mechanism often requires multiple iterations to identify an optimal read reference voltage value, resulting in the consumption of valuable reading time and consequent reduction in read performance. Furthermore, it is only used in the reading process and lacks the capability to alter the voltage distribution state of cells within the flash memory, nor can it mitigate the aging process inherent to NAND flash cell storage.

Although the two approaches could recover a certain of data, they have limitations and may cause some negative impact on system performance. ECC cannot correct errors over a threshold of 120 error bits per 1K code word. Furthermore, its correction time will increase as the number of error bits grows [8]. On the other hand, the read retry mechanism may severely impact the read performance when the flash memory requires more retries for error correction. In this paper, we propose a novel data recovery technique that is different from the previously mainstream data recovery approaches, called CellRejuvo. It cleverly leverages reprogramming techniques to alleviate cell aging resulting from charge loss while enhancing data reliability. Note that the term “reprogram” mentioned in this

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1077-3/24/10...\$15.00
<https://doi.org/10.1145/3676536.3676703>

article refers to the operation of writing the originally programmed wordline, not the “second program” step in two-step programming used in TLC or QLC devices [12, 17]. Additionally, in contrast to traditional out-place-update approaches [16], the paper proposed by Yu Cai et al. employs in-place reprogramming technology [1]. However, this method was deemed too ideal and did not account for real SSD conditions which may result in some errors in 3D NAND flash. To the best of our knowledge, this paper is the pioneer for exploration in the field of 3D NAND flash, discussing the application of reprogramming to enhance data robustness from a practical standpoint. The experimental results show that the error rate can be reduced by up to 38.28% on average.

2 BACKGROUND AND MOTIVATION

2.1 Data Retention Issues on 3D NAND Flash

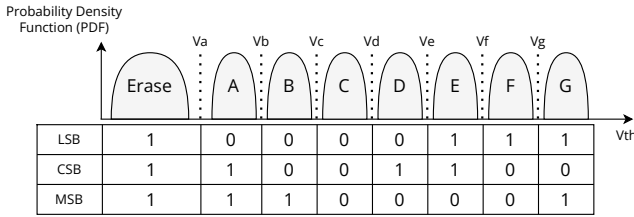


Figure 1: States and Coding Rule of 3D NAND TLC

In 3D TLC NAND flash, each memory cell is divided into eight different states to store three-bit information. Due to the different manufacturing vendors, the coding rules are not the same because of different Gray code encoding compositions. Figure 1 shows one of the coding rules of the eight states, including Erase and A to G states of 3D TLC NAND Flash. For instance, in this figure, the A state indicates that the data in LSB, CSB, and MSB is 0, 1, and 1, respectively.

In Figure 1, observing the junction of 1 and 0, the data stored in the LSB page of 3D TLC NAND flash is determined by the threshold voltages V_a and V_e . In other words, when electrons are lost from either the A state or the E state, it leads to an increase in errors in the LSB page. The data in the CSB page is determined by V_b , V_d , and V_f . Consequently, when electrons are lost from the B, D, or F state, errors in the CSB page are exacerbated. Lastly, the data in the MSB page is determined by V_c and V_g . Thus, electron loss from the C or G state results in increased errors in the MSB page.

Based on our observations, there are three common error types in 3D NAND flash: retention errors, read disturbance errors, and program interference errors [1, 3]. Retention error arises due to the gradual loss of electrons stored in the oxidized insulating layer over time. Read disturbance error occurs when reading a specific cell. This can potentially lead to interference with the surrounding cells, resulting in changes in their charge states and subsequent errors. Program interference error is similar to read disturbance error, denoting errors from charge interference in surrounding cells during the writing process to a specific cell [5]. Moreover, the temperature is a significant aging factor for semiconductors, including

the 3D NAND flash in our study [6]. The relationship between temperature and retention time follows the Arrhenius equation [26] that uses high-temperature aging to simulate retention time, which is also applied in our experiments. Furthermore, the accumulated program/erase (P/E) cycle also causes more errors in 3D NAND flash. Each time a programming or erasing operation is performed, the memory cells are affected by the application of voltage and physical stress, resulting in gradual wear and degeneration of the cells. Consequently, this will make the aforementioned three error types more serious. Among these error types, retention error is the most influential factor in compromising the reliability of 3D NAND flash.

In order to resolve the data retention issues on 3D NAND flash, some approaches have been proposed by researchers. BeLDPC [28] discusses a method that leverages the bit error characteristics resulting from P/E cycles and retention times. This method designs an LDPC encoding strategy to reduce the number of decoding iterations. Additionally, Feng et al. [9] propose an approach involving an analysis of the threshold voltage distribution in 3D TLC NAND flash memory to optimize read reference voltage prediction. However, both these ECC-based and read retry methods serve as remedial measures for data retention. They lack the ability to influence the voltage distribution within the NAND flash cell, thereby falling short of addressing the root cause of the retention issue.

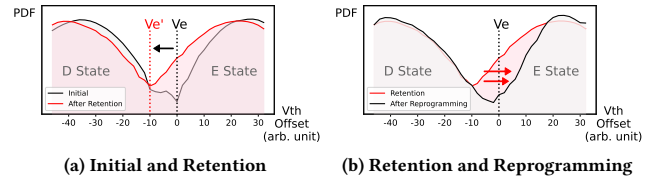


Figure 2: Shift of Threshold Voltage Distribution

To verify the actual data retention phenomenon and the impact of reprogramming of 3D TLC NAND flash, we performed a series of experiments on a real SSD platform (see Figure 12). In our experiments, we picked the block under 9000 P/E cycles and set the retention time to two months. Figure 2 illustrates the shift of threshold voltage distribution on a real-device test using the E state as an example, showing the impact of retention and reprogramming on voltage distribution. In Figure 2a, it is evident that cells originally in the E state lose electrons after two months of retention. This causes their voltage distribution to shift to the left side of V_e (i.e., V_e'). As a result, these cells, which originally represent the E state, are misclassified as the D state, leading to errors. At this point, the flash will activate the read retry mechanism, which involves adjusting the read reference voltage to V_e' to reduce errors. However, this process is time-consuming due to the unexpected read retries. Figure 2b demonstrates our preliminary experiments to evaluate the impact on the E state when writing the same pattern to the flash after two months of retention. We observed that cells originally shifted to the left can be charged by the program voltage, almost returning to their normal position. However, simply writing the same pattern to the flash will cause damage to the LSB and CSB pages of

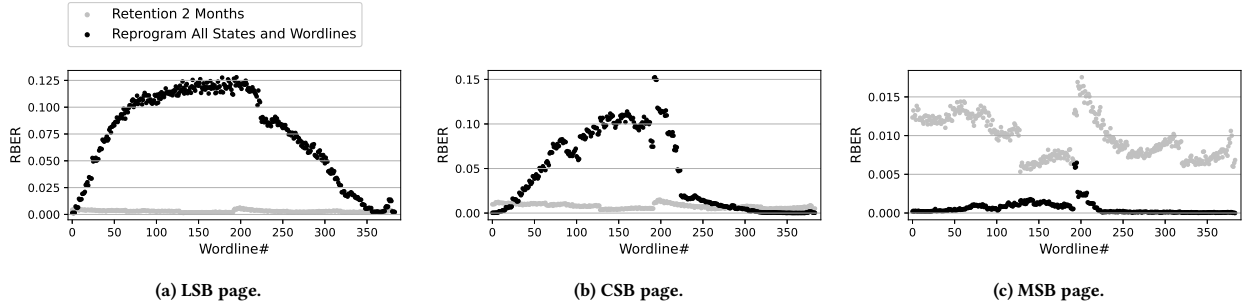


Figure 3: Reprogram Result for Different Page Types.

TLC, resulting in more severe program interference. Therefore, the flash requires intelligent approaches for reprogramming to become an effective data recovery method.

2.2 Motivation

Recently, many reprogramming techniques have been used for data sanitization with the growth of the importance of security on 3D NAND flash [7, 24]. Scrubbing [13] is one of the data-cleaning methods that reprograms the data to the highest electric potential state. Moreover, Wang et al. [25] introduce a reprogramming solution for data sanitization. It reprograms user data from the original state to other states, which cleans the target data and improves the read performance of 3D NAND flash.

However, the aforementioned reprogramming methods are mainly focused on data destruction aspects. Upon our investigation, it is possible to rethink the usage of reprogramming for data recovery. Figure 3 shows our preliminary experiments on reprogramming different types of page on the real-world 3D TLC NAND flash. The three figures represent the data of the LSB page, CSB page, and MSB page, respectively. The x-axis corresponds to the wordline number within a block under 9,000 P/E cycles, while the y-axis represents the Raw Bit Error Rate (RBER) measured under each wordline. The gray points represent the RBER values obtained after a 2-month device retention, whereas black points represent the RBER measured after the device underwent a 2-month retention period followed by reprogramming using the same pattern. It is evident that data errors in the LSB and CSB pages increased significantly after reprogramming. Surprisingly, errors in the MSB page exhibited a remarkable reduction. This has motivated us to explore the concept of reprogramming from a novel perspective: “data recovery”.

In various studies, including our preliminary experiments, it has been observed that the G state is the most susceptible to shifting. In other words, the G state is also the most error-prone, resulting in a higher incidence of errors [4]. These errors manifest primarily on the MSB page. As shown in Figure 4, for a block with 0 P/E cycle, after 12 months of retention, the G state experiences an offset of -16. In contrast, the offsets for other states do not exceed -10 at most. Consequently, the proposed method primarily focuses on reprogramming the error-prone G state to effectively reduce errors on the MSB page.

To the best of our knowledge, this paper marks the initial endeavor to employ reprogramming as a method for rescuing erroneous data, focusing on implementation aspects. The technical challenges fall on how to develop an intelligent reprogramming method that reduces errors on MSB pages with a minimal impact on LSB and CSB pages.

3 CELLREJUVO: RESCUING 3D NAND FLASH CELLS WITH REPROGRAMMING

We introduce the CellRejuvo strategy to address the data retention challenges observed in 3D TLC NAND flash. In this section, we begin by elucidating the write process of 3D NAND flash, which can provide a foundational understanding of the CellRejuvo principle. Subsequently, we delve into how CellRejuvo specifically targets the reprogramming of the G state. Then, we propose three variations of CellRejuvo based on the reprogrammed wordline density within a block, ranging from dense to loose. These three reprogramming methods are Dense Cell Rejuvenation Design (DCRD), Sparse Cell Rejuvenation Design (SCRD) type-1, and type-2. Lastly, we present the complete CellRejuvo workflow using an algorithmic approach for explanation.

3.1 CellRejuvo

In 3D TLC NAND flash, a write operation involves three pages, namely the LSB page, CSB page, and MSB page, which collectively form a wordline of data. The flash controller determines the appropriate state for a given cell based on the various data arrangements and combinations of these three pages. Additionally, in practice, upon receiving data from the host, the flash controller undergoes a scrambling process to ensure that the eight states occur at the same frequency, thus preventing lateral charge migration.

As illustrated in the schematic diagram in the upper part of Figure 5, each page has a size of 16 KB, equivalent to 131,072 bits. To illustrate, let us consider the first five bits. Initially, cells 1, 2, 3, 4, and 5 are all in the Erase state (ER). After programming, as illustrated in the lower part of Figure 5, their electric potentials are adjusted to states D, G, C, F, and B, respectively. Once all the cells within a wordline have their electric potentials programmed to the specified positions, and a write operation concludes, the interior of this wordline resembles the scenario depicted in Figure 1.

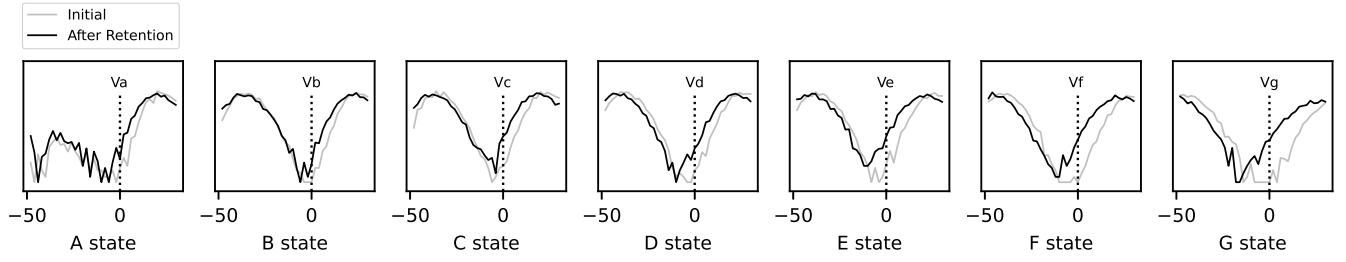


Figure 4: Distribution Offset among Different States.

As mentioned in Section 2, since G state is the most error-prone state in 3D TLC NAND flash, our proposed method will only reprogram cells that store G state in the original user data. The operational procedure for our design is as follows:

- (1) Initially, the target wordline to be reprogrammed is read.
- (2) Subsequently, a filter is employed to analyze the read data, identifying the positions corresponding to the G state.
- (3) Necessary data for the reprogram process are then prepared.
- (4) The prepared information is utilized to perform reprogramming on the target wordline.

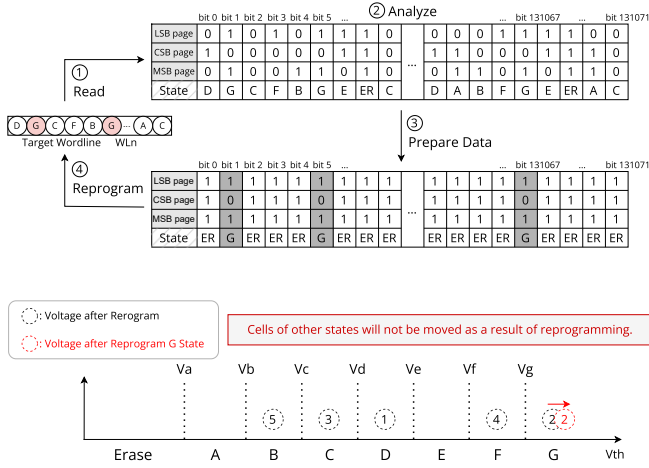


Figure 5: Process Flow and Effect Diagram of CellRejuvo

The example is illustrated in Figure 5. Upon analysis, it was determined that the G state positions corresponded to bit 1, bit 5, and so on until bit 131,067 (②). These positions were consequently set to the data intended for reprogramming, which constituted G state data (i.e., LSB-CSB-MSB being 1-0-1), while other positions were configured with Erase (ER) state data (i.e., LSB-CSB-MSB being 1-1-1). As per this approach, only bit 1, bit 5, and so on until bit 131,067 of the data prepared for reprogramming would be retained, with all other data bits set to ER state (③). After the data for reprogramming was prepared, the programming operation (④) could proceed. As data was exclusively written to the G state positions,

cells with G state storage electric potentials were charged, while other cells remained at their original voltage levels because they were programmed in the ER state, meaning they would not be actively charged due to the programming operation. This is depicted in the lower part of Figure 5, using the first five cells as an example. Cells 1, 2, 3, 4, and 5 initially stored data in states D, G, C, F, and B, respectively. After the reprogramming operation targeting the G state, only cell 2, which originally stored G state data, was shifted to the right, while the other cells remained unaffected.

Note that although cells storing other states may not be actively charged during the reprogramming operation, they can still be influenced by program interference from adjacent G state cells. As illustrated in Figure 6, program interference effects can extend to the cells surrounding those storing G state data, causing a slight charge. Therefore, not only the cells that store G state can improve the integrity of the data, but also the cells that store other states.

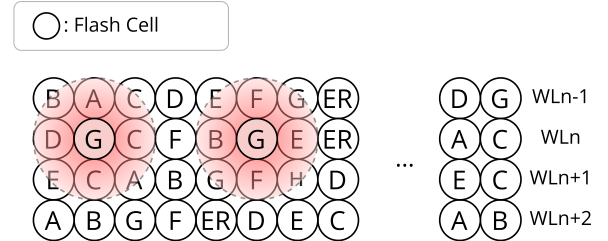


Figure 6: Program Interference of Reprogramming G State

Reprogramming the G state offers several advantages. Firstly, the G state is located as the last state in 3D NAND flash memory, which ensures that reprogramming it will not push the target cells beyond the next threshold voltage during charging, thereby preventing errors. Secondly, because the voltage transition from programming to the G state is larger than that from programming A to F states, the program interference caused by reprogramming the G state has the most significant impact, helping to rejuvenate other cells. Thirdly, in our preliminary experiments, we also evaluated data error conditions when reprogramming different states. In the experiment depicted in Figure 7, we tested blocks under 6,000, 9,000, and 12,000 P/E cycles, followed by reprogramming using the SCRD method (Section 3.1.2). Each state was tested independently. The final results demonstrate that for a block under 12,000 P/E cycles,

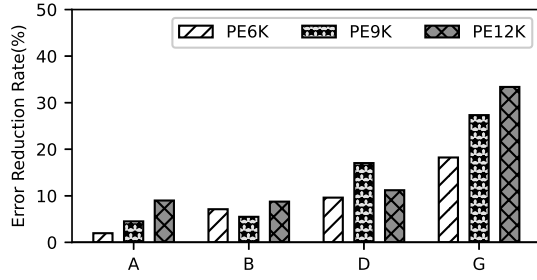


Figure 7: Effect of Reprogramming Different States on Data Error Rates

after a 2-month retention period, reprogramming the G state alone reduced the data error rate by 33.38%. In contrast, reprogramming A, B, and D states resulted in more modest reductions of 8.99%, 8.75%, and 11.2%, respectively. Additionally, reprogramming states C, E, and F had no obvious impact on reducing data errors and are therefore not included in Figure 7.

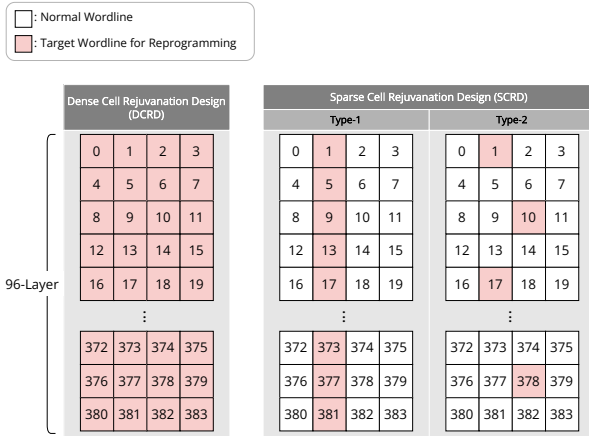


Figure 8: Different Reprogramming Methods

3.1.1 Dense Cell Rejuvenation Design (DCRD). Based on our detailed investigation, reprogramming the G state not only improves its data integrity, but also helps to rejuvenate other cells. As a result, we first propose a straightforward strategy to reprogram each wordline within the block with the aim of reducing the overall errors in the block, as depicted on the left side of Figure 8. This reprogramming approach is called “Dense Cell Rejuvenation Design (DCRD)”.

Figure 9 shows the initial error rate of LSB, CSB and MSB pages and the error rate after using DCRD for a block under 9,000 P/E cycles following a 6-month retention period. Based on the experiment results, we have made three key observations. Firstly, DCRD yields the most significant reduction in errors for MSB pages, with an average total RBER reduction of 23.41%. Secondly, DCRD also affects states B, D, and F controlled by CSB because of the program

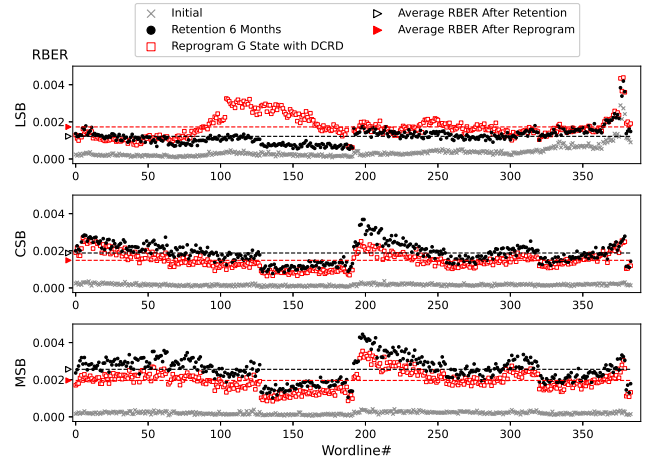


Figure 9: Initial and Error Rates Using DCRD

interference, leading to a notable 21% reduction in RBER for CSB pages. Finally, it is important to note that while DCRD effectively reduces errors in CSB and MSB pages, it has a negative impact on LSB pages. This is primarily because the electric potential difference between the ER state and the target reprogramming G state is the largest, and excessive program interference may cause some electrons in the ER state to exceed its adjacent threshold voltage V_a ; this may lead to misinterpretation as being in the A state, resulting in LSB page errors. In order to overcome the shortcomings of DCRD, we have developed another reprogramming method known as “Sparse Cell Rejuvenation Design (SCRD)”.

3.1.2 Sparse Cell Rejuvenation Design (SCRD). SCRD aims to reduce program interference among the cells to mitigate the shortcomings of DCRD that may increase LSB page errors. In contrast to DCRD, SCRD does not involve reprogramming every wordline. Instead, it tries to distribute the reprogramming operations among the wordline to strike a balance between data recovery and program interference. We have evaluated two different levels of distribution strategy among flash cell and proposed two types of SCRD: Type-1 SCRD and Type-2 SCRD. The main design idea of SCRD is illustrated on the right side of Figure 8. In our real device testing, each block comprises 96 layers, with every 4 wordlines forming a single layer. Type-1 SCRD selects the second wordline from each layer within the block for reprogramming. In contrast, type-2 SCRD exclusively targets the wordlines in even-numbered layers for a more dispersed reprogramming strategy. Figure 10 illustrates the original RBER and the RBER after recovery using SCRD for the same block, which had undergone 9,000 P/E cycles and experienced a 6-month retention period. In the SCRD type-1 experiment, the error reduction rates for LSB, CSB, and MSB pages were 19.49%, 25.91%, and 21.93%, respectively. In the SCRD type-2 experiment, the error reduction rates for LSB, CSB, and MSB pages were 11.45%, 19.71%, and 27.04%, respectively. It is important to note that our proposed method primarily targets the reduction of errors in the error-prone MSB pages while aiming to minimize its impact on

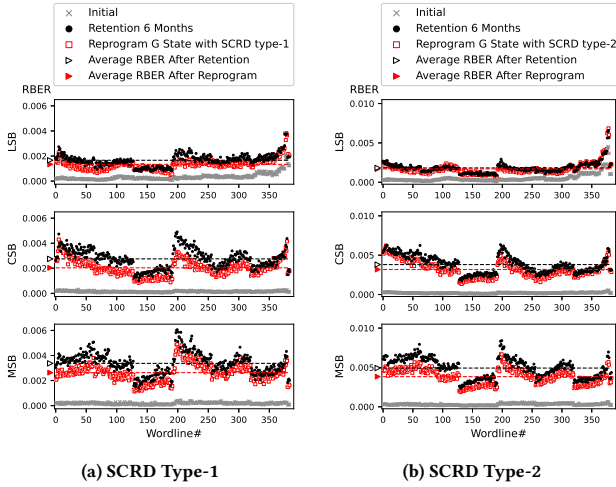


Figure 10: Initial and Recovered Error Rates using SCRD

LSB and CSB pages. As such, our SCRD approach yields excellent results, effectively reducing the RBER of MSB pages and offering the added benefit of error reduction for LSB and CSB pages.

In addition, we also analyzed the position within the block where the MSB page experiences the most significant error reduction. As illustrated in Figure 11, the area indicated by the darker shading exhibits an RBER reduction of over 25%. The first half of the entire block demonstrates a relatively concentrated trend, while the second half appears to have a more dispersed distribution. This suggests that when an MSB page is located in the first half of the block, it has a higher likelihood of being successfully restored through CellRejuvo in the event of errors caused by retention time. Furthermore, CellRejuvo not only diminishes errors within pages but also salvages many pages that might be seen as bad pages due to retention errors. Typically, contemporary ECC correction techniques can rectify under 120 bits of errors within a 1KiB codeword. When the errors within a page over 120 bits, the page is often classified as a bad page. In common bad block management strategies [20, 23], the entire block is labeled as a bad block, causing it unusable. However, with CellRejuvo, and considering a 25% error reduction, the page's error tolerance can be expanded to more bits. This not only significantly enhances the SSD's reliability but also minimizes the waste of space. An entire block does not need to be discarded due to a single problematic page.

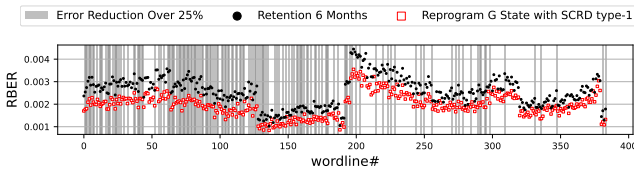


Figure 11: Error Reduction Analysis for MSB Pages

Algorithm 1: PrepareData

Input: WL : target wordline to reprogram.

Output: PD : a 16KB space to store data used for reprogramming G state.

```

1 Initialize  $PD, Data_{LSB}, Data_{CSB}, Data_{MSB} \leftarrow [0..0]$ 
2  $Data_{LSB} \leftarrow$  Read LSB Page of  $WL$ 
3  $Data_{CSB} \leftarrow$  Read CSB Page of  $WL$ 
4  $Data_{MSB} \leftarrow$  Read MSB Page of  $WL$ 
5 foreach bit do
6    $PD[i] \leftarrow \sim Data_{LSB}[i] \& Data_{CSB}[i] \& \sim Data_{MSB}[i]$ 
7 end
8 Return  $PD$ 

```

Algorithm 2: Different Reprogramming Methods for DCRD and SCRD in CellRejuvo

```

1 Initialize  $PD \leftarrow [0..0]$ : a 16KB space
2 for  $i = 0$  to 383 do // for DCRD
2-1 for  $i = 1$  to 383 step 4 do // SCRD type-1
2-2 for  $i = 1$  to 383 step  $\begin{cases} 9 & \text{if } i \text{ is odd} \\ 7 & \text{if } i \text{ is even} \end{cases}$  do // SCRD type-2
3    $PD \leftarrow PrepareData(WL_i)$ 
4   Program  $WL_i$  with the LSB, CSB, and MSB page data
   as  $[1..1], PD, [1..1]$ , respectively
5 end

```

3.2 Cell Rejuvenation Algorithms

When CellRejuvo is enabled, we analyze the positions within a wordline that inherently belong to the G state and how to prepare the data for reprogramming the G state. This algorithm called *PrepareData* corresponds to steps ①, ②, and ③ in process flow Figure 5. The algorithm takes WL as its input, representing which wordline requires data preparation. The output consists of a 16KB PD used for reprogramming. The process begins by initializing memory addresses and declaring four 16KB memory spaces for $PD, Data_{LSB}, Data_{CSB}$, and $Data_{MSB}$ (line 1). Subsequently, NAND flash page reading is initiated to retrieve the LSB, CSB, and MSB pages for WL_i , which are stored in $Data_{LSB}, Data_{CSB}$, and $Data_{MSB}$, respectively (lines 2-4). To identify the positions belonging to the G state, or, in other words, to locate positions where LSB, CSB, and MSB have the binary pattern 1-0-1, a simple bitwise operation is performed. This operation involves taking the bitwise complement (NOT operation) of $Data_{LSB}$ and $Data_{MSB}$, followed by executing the bitwise AND operation between these two datasets and $Data_{CSB}$ (line 6). Finally, this process returns the prepared data PD for reprogramming the G state (line 8).

Algorithm 2 illustrates the complete workflow of CellRejuvo. First, a 16KB memory space, referred to as PD , is initialized to store the data for reprogramming the G state (line 1). Next, choose either line 2, line 2-1, or line 2-2 based on the intended reprogramming method. It can be referred to Figure 8. As the DCRD method, we iterate through each WL_i within the block (line 2). For SCRD type-1 reprogramming (line 2-1), we select every second wordline in each

layer, starting with $i = 1$ and incrementing by 4 for each subsequent wordline. For SCRD type-2 reprogramming (line 2-2), we operate on wordlines based on whether i is odd or even. If i is odd, the next wordline position to be operated on is $i + 9$; if i is even, it is $i + 7$. Note that the iteration for i stops at 383 because, for our experimental device, a block has only 384 wordlines. During each iteration, we call *PrepareData* to obtain the required data (line 3). Finally, since a single program operation in 3D TLC NAND flash consists of three pages forming a wordline of data. We use the *PD* as the data for the CSB page while setting the LSB and MSB pages to contain all 1s. This combined data is then sent to the NAND flash controller for programming, achieving the reprogramming of the G state (line 4).

4 PERFORMANCE EVALUATION

4.1 Evaluation Metrics and Experiment Setup

To evaluate the capability of the proposed Cell Rejuvenation Design, we conduct a series of experiments with Microsoft MSR-Cambridge open workloads [18] on a real 3D NAND flash, as shown in Figure 12. The selection of MSR workloads is motivated by its widespread use in performance evaluations across various research papers [14, 17]. We selected blocks under 6,000, 9,000, and 12,000 P/E cycles for evaluation. Each P/E cycle group includes four blocks, resulting in the participation of a total of 12 physical blocks in the experiments. The intensive experiments are focused on evaluating the error reduction and read performance of Baseline, Dense, Sparse Type-1, and Sparse Type-2. The Baseline represents the results obtained

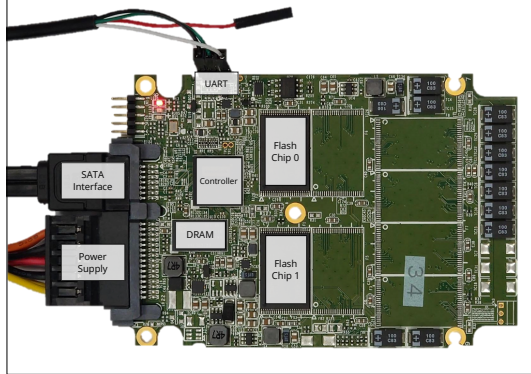


Figure 12: SSD Development Platform

without employing any data recovery technique. Dense records the recovered RBER achieved by DCRD method after each retention period, and the same goes for Sparse Type-1 and Sparse Type-2. For measuring error reduction, we employed retention periods of 6 months, 9 months, 12 months, 24 months, and 36 months as our standards. Finally, the data collected from all 12 blocks were averaged to obtain the final results. On the performance measurements, we gathered data from the 12 selected blocks to derive the retention scenarios at different ages: early stage (6 months), middle stage (12 months), and late stage (36 months). It's important to note that this measurement does not account for internal operations within the FTL, such as garbage collection or wear leveling.

4.2 Error Reduction

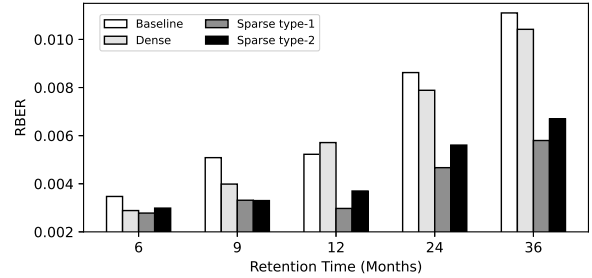


Figure 13: RBER using Different Reprogramming Methods under Various Retention Times

Figure 13 shows the RBER for each reprogramming method at various retention times. At each retention time, the state inherits the previous period. For instance, the data for “9-month Dense” is derived from the “6-month Dense” data after an additional 3 months of retention and one subsequent DCRD reprogramming operation. This approach allows us to simulate the effects of continuous usage with different methods.

We observed several key observations in this figure. Firstly, the error status of the Baseline method is the worst during most of the retention period. It is obvious that the proposed DCRD and SCRD significantly reduce the RBER of 3D NAND flash. As shown in the figure, the proposed methods become more effective with the increased retention time. However, at the 12-month point, the error status of Dense is slightly higher than Baseline. This is due to the phenomenon mentioned in Section 3.1.1, where using DCRD induces some errors in the LSB pages, resulting in a worse error condition compared to Baseline. Secondly, we noticed that during the early stages of retention (6-month and 9-month), the error status of Sparse does not significantly differ from the other methods. However, as retention progresses into the middle and late stages, the impact gradually becomes more pronounced. Taking Sparse type-1 as an example, in the early stages of retention, the RBERs at 6-month and 9-month are 19.9% and 34.8% lower than Baseline, respectively. In the middle and late stages of retention, the RBERs at 12-month, 24-month, and 36-month are 43.1%, 45.8%, and 47.8% lower than Baseline, respectively. Lastly, there is not much difference between Sparse type-1 and type-2 during the early stages of retention. However, in the middle and late stages of retention, it's evident that type-1 is better than type-2. In our specific device context, the results of reprogramming 96 wordlines (Sparse type-1) in one block yield better results than reprogramming 48 wordlines (Sparse type-2). Furthermore, Sparse type-1 manages to restore more cells to their correct potential at 12-month, causing the RBER to decrease even further compared to the 6-month period.

The proposed DCRD and SCRD methods clearly reduce RBER in 3D NAND flash, with their effectiveness increasing as retention time extends. In particular, the Sparse Type-1 of SCRD is better than DCRD and Sparse Type-2 which reduces more than 3.62% and 6.9% RBERs at 6-month retention time, respectively. Moreover, in the middle and late stages of retention, the RBERs at 12-month,

24-month, and 36-month are 19.59%, 16.74%, and 13.39% lower than the Sparse Type-2, respectively. Therefore, it can be inferred that the number of wordlines to be reprogrammed should not be too many or too few.

Between DCRD and SCRD, we can conclude that SCRD reprogramming yields the best results. However, when comparing SCRD type-1 and SCRD type-2, there is not a one-size-fits-all solution. This is because, even though SCRD type-1 offers superior error reduction, it comes at the cost of performance and energy overhead due to reprogramming additional wordlines. Therefore, when choosing between the two, it is important to consider certain specific requirements and strike a balance between error reduction effectiveness and the associated performance and energy overhead.

4.3 Read Performance

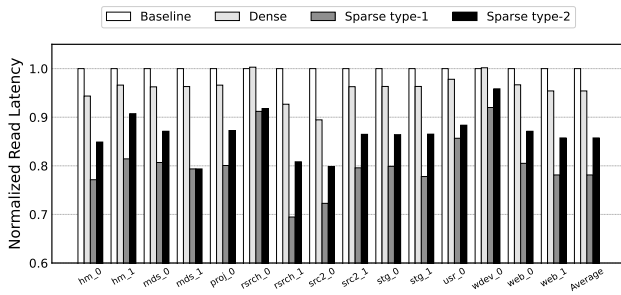


Figure 14: Read Latency using Different Reprogramming Methods in Late Stage

Figure 14 shows the read latency while applying various reprogramming methods under different workloads in the late stage. The y-axis is normalized to the Baseline. The experiment results show that the proposed DCRD and SCRD results in a much shorter read latency than the Baseline. This is because when read requests need to access data with more than 120 error bits per 1KiB in the NAND flash, the read retry mechanism is initiated. Read retry attempts for sets of parameter provided by the manufacturer until ECC correction is either unsuccessful or successful for a specific parameter set, which results in heavy processing time. Although our DCRD and SCRD may also encounter situations requiring read retries, the experimental results revealed that the processing time of read requests while applying the proposed Sparse Type-1 is 21.86% faster than the Baseline on average and even up to 30.53% faster than the Baseline on rsrc_1. However, in rsrc_0 and wdev_0, because the addresses accessed by workloads happen to be in the LSB pages of most blocks, DCRD will cause some errors to occur in LSB pages, causing Dense to have slightly worse performance under these two workloads. In summary, the proposed CellRejuvo not only accelerates ECC decoding due to its outstanding error reduction capabilities but also reduces the necessity for read retries, resulting in a substantial enhancement in read performance.

5 CONCLUSION

In this paper, we propose a novel data recovery strategy called CellRejuvo to rescue aging 3D TLC NAND flash cells. Differing from time-consuming and constrained approaches like ECC and read retry, our proposed CellRejuvo addresses the retention issue at its root. The reprogramming method, traditionally employed for data destruction, is leveraged here to achieve data recovery. To the best of our knowledge, this paper is the pioneer to explore, using real SSD device, the application of reprogramming techniques to mitigate data errors. Specifically, CellRejuvo achieves the greatest error reduction by reprogramming the most error-prone G state in 3D TLC NAND flash. We implemented CellRejuvo on a real SSD device, and the experiments show that the method proposed in this article can reduce errors by 38.28% compared to the Baseline on average, and significantly improve read performance by 21.86% in the late stage.

ACKNOWLEDGMENTS

This work was supported in part by National Science and Technology Council under grant nos. 111-2221-E-011-087-MY3, 112-2222-E-011-010-MY3, 113-2223-E-001-001, 111-2221-E-001-013-MY3, 113-2927-I-001-502, and 111-2923-E-002-014-MY3 and Academia Sinica under grant no. AS-IA-111-M01.

REFERENCES

- [1] Yu Cai, Erich F Haratsch, Onur Mutlu, and Ken Mai. 2012. Error patterns in MLC NAND flash memory: Measurement, characterization, and analysis. In *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 521–526.
- [2] Yu Cai, Erich F Haratsch, Onur Mutlu, and Ken Mai. 2013. Threshold voltage distribution in MLC NAND flash memory: Characterization, analysis, and modeling. In *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1285–1290.
- [3] Yu Cai, Yixin Luo, Saugata Ghose, and Onur Mutlu. 2015. Read disturb errors in MLC NAND flash memory: Characterization, mitigation, and recovery. In *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 438–449.
- [4] Yu Cai, Yixin Luo, Erich F. Haratsch, Ken Mai, Saugata Ghose, and Onur Mutlu. 2018. Experimental Characterization, Optimization, and Recovery of Data Retention Errors in MLC NAND Flash Memory. arXiv:1805.02819 [cs.AR]
- [5] Yu-Ming Chang, Yuan-Hao Chang, Tei-Wei Kuo, Yung-Chun Li, and Hsiang-Pang Li. 2015. Disturbance relaxation for 3D flash memory. *IEEE Trans. Comput.* 65, 5 (2015), 1467–1483.
- [6] Yi-Shen Chen and Peng-Sheng Chen. 2016. A software-based redundant execution programming model for transient fault detection and correction. In *2016 45th International Conference on Parallel Processing Workshops (ICPPW)*. IEEE, 66–71.
- [7] Shih-Chun Chou, Yi-Shen Chen, Ping-Xiang Chen, Yuan-Hao Chang, Ming-Chang Yang, Tei-Wei Kuo, Yu-Fang Chen, and Yu-Ming Chang. 2023. FSD: File-related Secure Deletion to Prolong the Lifetime of Solid-State Drives. In *2023 IEEE 12th Non-Volatile Memory Systems and Applications Symposium (NVMISA)*. IEEE, 19–24.
- [8] Guiqiang Dong, Ningde Xie, and Tong Zhang. 2010. On the use of soft-decision error-correction codes in NAND flash memory. *IEEE Transactions on Circuits and Systems I: Regular Papers* 58, 2 (2010), 429–439.
- [9] Hua Feng, Debao Wei, Yongchao Wang, Yu Song, Zhelong Piao, and Liyan Qiao. 2023. Lightweight Read Reference Voltage Calibration Strategy for Improving 3-D TLC NAND Flash Memory Reliability. *IEEE Transactions on Device and Materials Reliability* (2023).
- [10] George Forney. 1965. On decoding BCH codes. *IEEE Transactions on information theory* 11, 4 (1965), 549–557.
- [11] Robert Gallager. 1962. Low-density parity-check codes. *IRE Transactions on information theory* 8, 1 (1962), 21–28.
- [12] Congming Gao, Min Ye, Chun Jason Xue, Youtao Zhang, Liang Shi, Jiwu Shu, and Jun Yang. 2022. Reprogramming 3D TLC Flash Memory based Solid State Drives. In *ACM Transactions on Storage*, Vol. 18. ACM, 1–33. <https://doi.org/10.1145/3487064>
- [13] Md Mehedi Hasan and Biswajit Ray. 2020. Data Recovery from “Scrubbed” NAND Flash Storage: Need for Analog Sanitization. In *29th USENIX Security Symposium*

- (USENIX Security 20). USENIX Association, 1399–1408. <https://www.usenix.org/conference/usenixsecurity20/presentation/hasan>
- [14] Duwon Hong, Myungsuk Kim, Geonhee Cho, Dusol Lee, and Jihong Kim. 2022. {GuardedErase}: Extending {SSD} Lifetimes by Protecting Weak Wordlines. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 133–146.
 - [15] Jen-Wei Hsieh, Li-Pin Chang, and Tei-Wei Kuo. 2005. Efficient on-line identification of hot data for flash-memory management. In *Proceedings of the 2005 ACM symposium on Applied computing*. 838–842.
 - [16] Jen-Wei Hsieh, Tei-Wei Kuo, Po-Liang Wu, and Yu-Chung Huang. 2007. Energy-efficient and performance-enhanced disks using flash-memory cache. In *Proceedings of the 2007 international symposium on Low power electronics and design*. 334–339.
 - [17] Yina Lv, Liang Shi, Qiao Li, Congming Gao, Yunpeng Song, Longfei Luo, and Youtao Zhang. 2023. MGC: Multiple-Gray-Code for 3D NAND Flash based High-Density SSDs. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1–13. <https://doi.org/10.1109/HPCA56546.2023.10070946>
 - [18] Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron. 2008. Write off-loading: Practical power management for enterprise storage. *ACM Transactions on Storage (TOS)* 4, 3 (2008), 1–23.
 - [19] Jisung Park, Myungsuk Kim, Myoungjun Chun, Lois Orosa, Jihong Kim, and Onur Mutlu. 2021. Reducing Solid-State Drive Read Latency by Optimizing Read-Retry. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 702–716.
 - [20] Borja Peleato, Haleh Tabrizi, Rajiv Agarwal, and Jeffrey Ferreira. 2015. BER-based wear leveling and bad block management for NAND flash. In *2015 IEEE International Conference on Communications (ICC)*. IEEE, 295–300.
 - [21] Arul Subbiah and Tokunbo Ogunfunmi. 2019. A flexible hybrid BCH decoder for modern NAND flash memories using general purpose graphical processing units (GPGPUs). *Micromachines* 10, 6 (2019), 365.
 - [22] Che-Wei Tsao, Yuan-Hao Chang, Ming-Chang Yang, and Po-Chun Huang. 2015. Efficient victim block selection for flash storage devices. *IEEE Trans. Comput.* 64, 12 (2015), 3444–3460.
 - [23] Chundong Wang and Weng-Fai Wong. 2012. Extending the lifetime of NAND flash memory by salvaging bad blocks. In *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 260–263.
 - [24] Wei-Chen Wang, Chien-Chung Ho, Yuan-Hao Chang, Tei-Wei Kuo, and Ping-Hsien Lin. 2018. Scrubbing-aware secure deletion for 3-D NAND flash. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2790–2801.
 - [25] Wei-Chen Wang, Chien-Chung Ho, Yung-Chun Li, Liang-Chi Chen, and Yu-Ming Chang. 2023. Reaping Both Latency and Reliability Benefits With Elaborate Sanitization Design for 3D TLC NAND Flash. In *IEEE Transactions on Computers*. IEEE, 1–13. <https://doi.org/10.1109/TC.2023.3272280>
 - [26] M Xu, C Tan, and M Li. 2003. Extended Arrhenius law of time-to-breakdown of ultrathin gate oxides. *Applied Physics Letters* 82, 15, 2482–2484. <https://doi.org/10.1063/1.1566460>
 - [27] Ming-Chang Yang, Yuan-Hao Chang, Che-Wei Tsao, and Po-Chun Huang. 2013. New ERA: New efficient reliability-aware wear leveling for endurance enhancement of flash storage devices. In *Proceedings of the 50th Annual Design Automation Conference*. 1–6.
 - [28] Meng Zhang, Fei Wu, Qin Yu, Weihua Liu, Lanlan Cui, Yahui Zhao, and Changsheng Xie. 2020. BeLDPC: Bit errors aware adaptive rate LDPC codes for 3D TLC NAND flash memory. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 302–305.